

Swami Sahajanand College of Computer Science

B.C.A. SEM - VI

Subject: Web Application Development Using ASP.Net

UNIT 2

Introduction

- ◆ Master page & content page.
- ◆ Theme – CSS, Skin File.
- ◆ Validation Control.
- ◆ Ad Rotator.
- ◆ Navigation control – Menu, Tree view & Site map path.
- ◆ File upload and Image control.

Q.-1 What is Master Pages? Explain in detail.**Master Page:**

- ❑ ASP.NET master pages allow you to create a consistent layout for the pages in your application. A single master page defines the look and feel and standard behavior that you want for all of the pages (or a group of pages) in your application.
- ❑ You can then create individual content pages that contain the content you want to display.
- ❑ When users request the content pages, they merge with the master page to produce output that combines the layout of the master page with the content from the content page.
- ❑ Master pages actually consist of two pieces, the master page itself and one or more content pages.

Master Pages

- ❑ A master page is an ASP.NET file with the extension .master (for example, MySite.master) with a predefined layout that can include static text, HTML elements, and server controls. The master page is identified by a special @ Master directive that replaces the @ Page directive that is used for ordinary .aspx pages. The directive looks like the following.

```
<%@ Master Language="VB" %>
```
- ❑ The @ Master directive can contain most of the same directives that a @ Control directive can contain. For example, the following master-page directive includes the name of a code-behind file, and assigns a class name to the master page.

```
<%@ Master Language="VB" CodeFile="MasterPage.master.vb" Inherits="MasterPage" %>
```
- ❑ In addition to the @ Master directive, the master page also contains all of the top-level HTML elements for a page, such as html, head, and form.
- ❑ For example, on a master page you might use an HTML table for the layout, an img element for your company logo, static text for the copyright notice, and server controls to create standard navigation for your site.
- ❑ You can use any HTML and any ASP.NET elements as part of your master page.

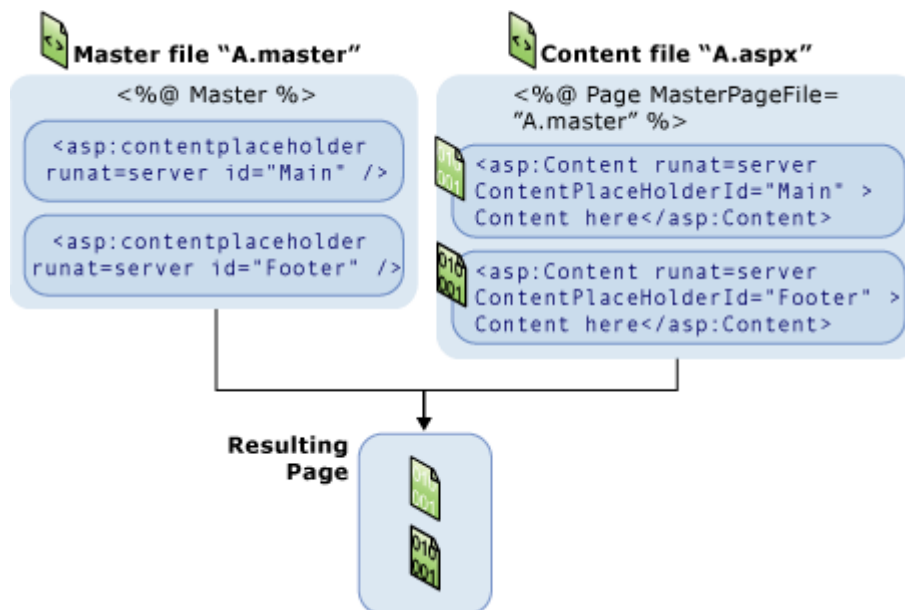
Replaceable Content Placeholders

- ❑ In addition to static text and controls that will appear on all pages, the master page also includes one or more ContentPlaceHolder controls.
- ❑ These placeholder controls define regions where replaceable content will appear. In turn, the replaceable content is defined in content pages.
- ❑ After you have defined the ContentPlaceHolder controls, a master page might look like the following.

Content Pages

- ❑ You define the content for the master page's placeholder controls by creating individual content pages, which are ASP.NET pages (.aspx files and, optionally, code-behind files) that are bound to a specific master page.
- ❑ The binding is established in the content page's @ Page directive by including a MasterPageFile attribute that points to the master page to be used. For example, a content page might have the following @ Page directive, which binds it to the Master1.master page.
- ❑ In the content page, you create the content by adding Content controls and mapping them to ContentPlaceHolder controls on the master page. For example, the master page might have content placeholders called Main and Footer.

- ❑ In the content page, you can create two Content controls, one that is mapped to the ContentPlaceHolder control Main and the other mapped to the ContentPlaceHolder control Footer, as shown in the following figure.
- ❑ Replacing placeholder content



- ❑ After creating Content controls, you add text and controls to them. In a content page, anything that is not inside the Content controls (except script blocks for server code) results in an error.
- ❑ You can perform any tasks in a content page that you do in an ASP.NET page. For example, you can generate content for a Content control using server controls and database queries or other dynamic mechanisms.
- ❑ A content page might look like the following.

```
<% @ Page Language="VB" MasterPageFile="~/Master.master" Title="Content Page
1" %>

<asp:Content ID="Content1" ContentPlaceHolderID="Main" Runat="Server">
    Main content.
</asp:Content>

<asp:Content ID="Content2" ContentPlaceHolderID="Footer" Runat="Server">
    Footer content.
</asp:content>
```

- ❑ The @ Page directive binds the content page to a specific master page, and it defines a title for the page that will be merged into the master page. Note that the content page contains no other markup outside of the Content controls.
- ❑ You can create multiple master pages to define different layouts for different parts of your site, and a different set of content pages for each master page.

Advantages of Master Pages

- ❑ Master pages provide functionality that developers have traditionally created by copying existing code, text, and control elements repeatedly; using framesets; using include files for common elements; using ASP.NET user controls; and so on.
- ❑ Advantages of master pages include the following:
- ❑ They allow you to centralize the common functionality of your pages so that you can make updates in just one place.

- ❑ They make it easy to create one set of controls and code and apply the results to a set of pages. For example, you can use controls on the master page to create a menu that applies to all pages.
- ❑ They give you fine-grained control over the layout of the final page by allowing you to control how the placeholder controls are rendered.
- ❑ They provide an object model that allows you to customize the master page from individual content pages.

Q.-2 What is Theme? Explain CSS in detail.

- ❑ A theme is a collection of property settings that allow you to define the look of pages and controls, and then apply the look consistently across pages in a Web application, across an entire Web application, or across all Web applications on a server.

Themes and Control Skins

- ❑ Themes are made up of a set of elements: skins, cascading style sheets (CSS), images, and other resources. At a minimum, a theme will contain skins.
- ❑ Themes are defined in special directories in your Web site or on your Web server.

Skins

- ❑ A skin file has the file name extension .skin and contains property settings for individual controls such as Button, Label, TextBox, or Calendar controls.
- ❑ Control skin settings are like the control markup itself, but contain only the properties you want to set as part of the theme.
- ❑ For example, the following is a control skin for a Button control:
`<asp:button runat="server" BackColor="lightblue" ForeColor="black" />`
- ❑ You create .skin files in the Theme folder. A .skin file can contain one or more control skins for one or more control types.
- ❑ You can define skins in a separate file for each control or define all the skins for a theme in a single file.
- ❑ There are two types of control skins, default skins and named skins:
- ❑ A default skin automatically applies to all controls of the same type when a theme is applied to a page. A control skin is a default skin if it does not have a SkinID attribute.
- ❑ For example, if you create a default skin for a Calendar control, the control skin applies to all Calendar controls on pages that use the theme. (Default skins are matched exactly by control type, so that a Button control skin applies to all Button controls, but not to LinkButton controls or to controls that derive from the Button object.)
- ❑ A named skin is a control skin with a SkinID property set.
- ❑ Named skins do not automatically apply to controls by type. Instead, you explicitly apply a named skin to a control by setting the control's SkinID property.
- ❑ Creating named skins allows you to set different skins for different instances of the same control in an application.

Cascading Style Sheets

- ❑ A theme can also include a cascading style sheet (.css file). When you put a .css file in the theme folder, the style sheet is applied automatically as part of the theme.
- ❑ You define a style sheet using the file name extension .css in the theme folder.

Theme Graphics and Other Resources

- ❑ Themes can also include graphics and other resources, such as script files or sound files. For example, part of your page theme might include a skin for a TreeView control. As part

of the theme, you can include the graphics used to represent the expand button and the collapse button.

- ❑ Typically, the resource files for the theme are in the same folder as the skin files for that theme, but they can be elsewhere in the Web application, in a subfolder of the theme folder for example.
- ❑ To refer to a resource file in a subfolder of the theme folder, use a path like the one shown in this Image control skin:
`<asp:Image runat="server" ImageUrl="ThemeSubfolder/filename.ext" />`
- ❑ You can also store your resource files outside the theme folder.
- ❑ If you use the tilde (~) syntax to refer to the resource files, the Web application will automatically find the images.
- ❑ For example, if you place the resources for a theme in a subfolder of your application, you can use paths of the form ~/SubFolder/filename.ext to refer to resource files, as in the following example.

`<asp:Image runat="server" ImageUrl="~/AppSubfolder/filename.ext" />`

Scoping Themes

- ❑ You can define themes for a single Web application, or as global themes that can be used by all applications on a Web server.
- ❑ After a theme is defined, it can be placed on individual pages using the Theme or StyleSheetTheme attribute of the @ Page directive, or it can be applied to all pages in an application by setting the <pages> element in the application configuration file.
- ❑ If the <pages> element is defined in the Machine.config file, the theme will apply to all pages in Web applications on the server.

Page Themes

- ❑ A page theme is a theme folder with control skins, style sheets, graphics files and other resources created as a subfolder of the \App_Themes folder in your Web site.
- ❑ Each theme is a different subfolder of the \App_Themes folder. The following example shows a typical page theme, defining two themes named BlueTheme and PinkTheme.

```
MyWebSite
  App_Themes
    BlueTheme
      Controls.skin
      BlueTheme.css
    PinkTheme
      Controls.skin
      PinkTheme.css
  Global Themes
```

- ❑ A global theme is a theme that you can apply to all the Web sites on a server. Global themes allow you to define an overall look for your domain when you maintain multiple Web sites on the same server.
- ❑ Global themes are like page themes in that they include property settings, style sheet settings, and graphics.
- ❑ However, global themes are stored in a folder named Themes that is global to the Web server. Any Web site on the server, and any page in any Web site, can reference a global theme.

Theme Settings Precedence

- ❑ You can specify the precedence that theme settings take over local control settings by specifying how the theme is applied.

- ❑ If you set a page's Theme property, control settings in the theme and the page are merged to form the final settings for the control. If a control setting is defined in both the control and the theme, the control settings from the theme override any page settings on the control. .
- ❑ This strategy enables the theme to create a consistent look across pages, even if controls on the pages already have individual property settings.
- ❑ For example, it allows you to apply a theme to a page you created in an earlier version of ASP.NET.
- ❑ Alternatively, you can apply a theme as a style sheet theme by setting the page's StyleSheetTheme property.
- ❑ In this case, local page settings take precedence over those defined in the theme when the setting is defined in both places.
- ❑ This is the model used by cascading style sheets. You might apply a theme as a style sheet theme if you want to be able to set the properties of individual controls on the page while still applying a theme for an overall look.
- ❑ Global theme elements cannot be partially replaced by elements of application-level themes.
- ❑ If you create an application-level theme with the same name as a global theme, theme elements in the application-level theme will not override the global theme elements.
- ❑ Properties You Can Define Using Themes
- ❑ As a rule, you can use themes to define properties that concern a page or control's appearance or static content.
- ❑ You can set only those properties that have a ThemeableAttribute attribute set to true in the control class.
- ❑ Properties that explicitly specify control behavior rather than appearance do not accept theme values.
- ❑ For example, you cannot set a Button control's CommandName property by using a theme. Similarly, you cannot use a theme to set a GridView control's AllowPaging property or DataSource property.
- ❑ Note that you cannot use expression builders, which generate code expressions for assignment in a page at compile time, in themes or skins.

Themes vs. Cascading Style Sheets

- ❑ Themes are similar to cascading style sheets in that both themes and style sheets define a set of common attributes that can be applied to any page. However, themes differ from style sheets in the following ways:
- ❑ Themes can define many properties of a control or page, not just style properties. For example, using themes, you can specify the graphics for a TreeView control, the template layout of a GridView control, and so on.
- ❑ Themes can include graphics.
- ❑ Themes do not cascade the way style sheets do. By default, any property values defined in a theme referenced by a page's Theme property override the property values declaratively set on a control, unless you explicitly apply the theme using the StyleSheetTheme property. For more information, see the Theme Settings Precedence section above.
- ❑ Only one theme can be applied to each page. You cannot apply multiple themes to a page, unlike style sheets where multiple style sheets can be applied.

Security Considerations

- ❑ Themes can cause security issues when they are used on your Web site. Malicious themes can be used to: Alter a control's behavior so that it does not behave as expected.
- ❑ Inject client-side script, therefore posing a cross-site scripting risk.
- ❑ Alter validation. Expose sensitive information.
- ❑ The mitigations for these common threats are:
- ❑ Protect the global and application theme directories with proper access control settings. Only trusted users should be allowed to write files to the theme directories.
- ❑ Do not use themes from an untrusted source. Always examine any themes from outside your organization for malicious code before using them on you Web site.
- ❑ Do not expose the theme name in query data. Malicious users could use this information to use themes that are unknown to the developer and thereby expose sensitive information.

Q.-3. Explain Validation Control in detail.

Required Field Validator:

- ❑ The RequiredFieldValidator control is used to make an input control a required field.
- ❑ With this control, the validation fails if the input value does not change from its initial value. By default, the initial value is an empty string ("").
- ❑ Leading and trailing spaces of the input value are removed before validation.
- ❑ The InitialValue property does not set the default value for the input control. It indicates the value that you do not want the user to enter in the input control.

Properties

Property	Description
BackColor	The background color of the RequiredFieldValidator control
ControlToValidate	The id of the control to validate
Display	The display behavior for the validation control. Legal values are: • None (the control is not displayed. Used to show the error message only in the ValidationSummary control) • Static (the control displays an error message if validation fails. Space is reserved on the page for the message even if the input passes validation. • Dynamic (the control displays an error message if validation fails. Space is not reserved on the page for the message if the input passes validation)
EnableClientScript	A Boolean value that specifies whether client-side validation is enabled or not
Enabled	A Boolean value that specifies whether the validation control is enabled or not
ErrorMessage	The text to display in the ValidationSummary control when validation fails. Note: This text will also be displayed in the validation control if the Text property is not set
ForeColor	The foreground color of the control
Id	A unique id for the control
InitialValue	Specifies the starting value of the input control. Default value is ""
IsValid	A Boolean value that indicates whether the control specified by ControlToValidate is determined to be valid

Runat	Specifies that the control is a server control. Must be set to "server"
Text	The message to display when validation fails

Range Validator:

- ❑ The RangeValidator control is used to check that the user enters an input value that falls between two values. It is possible to check ranges within numbers, dates, and characters.
- ❑ The validation will not fail if the input control is empty. Use the RequiredFieldValidator control to make the field required.
- ❑ The validation will not fail if the input value cannot be converted to the data type specified. Use the CompareValidator control, with its Operator property set to ValidationCompareOperator.DataTypeCheck, to verify the data type of the input value.

Properties

Property	Description
BackColor	The background color of the RangeValidator control
ControlToValidate	The id of the control to validate
Display	The display behavior for the validation control. Legal values are: • None (the control is not displayed. Used to show the error message only in the ValidationSummary control) • Static (the control displays an error message if validation fails. Space is reserved on the page for the message even if the input passes validation. • Dynamic (the control displays an error message if validation fails. Space is not reserved on the page for the message if the input passes validation)
EnableClientScript	A Boolean value that specifies whether client-side validation is enabled or not
Enabled	A Boolean value that specifies whether the validation control is enabled or not
ErrorMessage	The text to display in the ValidationSummary control when validation fails. Note: This text will also be displayed in the validation control if the Text property is not set
ForeColor	The foreground color of the control
Id	A unique id for the control
IsValid	A Boolean value that indicates whether the control specified by ControlToValidate is determined to be valid
MaximumValue	Specifies the maximum value of the input control
MinimumValue	Specifies the minimum value of the input control
Runat	Specifies that the control is a server control. Must be set to "server"
Type	Specifies the data type of the value to check. The types are: • Currency • Date • Double • Integer • String
Text	The message to display when validation fails

Regular Expression Validator:

- ❑ The RegularExpressionValidator control is used to ensure that an input value matches a specified pattern.
- ❑ Both server- and client-side validation are performed unless the browser does not support client-side validation or the EnableClientScript property is set to false.
- ❑ The validation will not fail if the input control is empty. Use the RequiredFieldValidator control to make the field required.

Properties

Property	Description
BackColor	The background color of the RegularExpressionValidator control
ControlToValidate	The id of the control to validate
Display	The display behavior for the validation control. Legal values are: • None (the control is not displayed. Used to show the error message only in the ValidationSummary control) • Static (the control displays an error message if validation fails. Space is reserved on the page for the message even if the input passes validation. • Dynamic (the control displays an error message if validation fails. Space is not reserved on the page for the message if the input passes validation)
EnableClientScript	A Boolean value that specifies whether client-side validation is enabled or not
Enabled	A Boolean value that specifies whether the validation control is enabled or not
ErrorMessage	The text to display in the ValidationSummary control when validation fails. Note: This text will also be displayed in the validation control if the Text property is not set
ForeColor	The foreground color of the control
Id	A unique id for the control
IsValid	A Boolean value that indicates whether the control specified by ControlToValidate is determined to be valid
Runat	Specifies that the control is a server control. Must be set to "server"
BackColor	The background color of the RegularExpressionValidator control
Text	The message to display when validation fails
ValidationExpression	Specifies the expression used to validate input control. The expression validation syntax is different on the client than on the server. JScript is used on the client. On the server, the language you have specified is use

Custom Validation:

- ❏ The CustomValidator control allows you to write a method to handle the validation of the value entered.

Properties

Property	Description
BackColor	The background color of the CustomValidator control
ClientValidationFunction	Specifies the name of the client-side validation script function to be executed. Note: The script must be in a language that the browser supports, such as VBScript or JScript With VBScript, the function must be in the form: Sub FunctionName (source, arguments) With JScript, the function must be in the form: Function FunctionName (source, arguments)
ControlToValidate	The id of the control to validate
Display	The display behavior for the validation control. Legal values are: • None (the control is not displayed. Used to show the error message only in the ValidationSummary control) • Static (the control displays an error message if validation fails. Space is reserved on the page for the message even if the input passes validation. • Dynamic (the control displays an error message if validation fails. Space is not reserved on the page for the message if the input passes validation)
EnableClientScript	A Boolean value that specifies whether client-side validation is enabled or not
Enabled	A Boolean value that specifies whether the validation control is enabled or not
ErrorMessage	The text to display in the ValidationSummary control when validation fails. Note: This text will also be displayed in the validation control if the Text property is not set
ForeColor	The foreground color of the control
Id	A unique id for the control
IsValid	A Boolean value that indicates whether the control specified by ControlToValidate is determined to be valid
OnServerValidate	Specifies the name of the server-side validation script function to be executed
Runat	Specifies that the control is a server control. Must be set to "server"
Text	The message to display when validation fails

Example:**Validation Summary:**

- ❑ The ValidationSummary control is used to display a summary of all validation errors occurred in a Web page.
- ❑ The error message displayed in this control is specified by the ErrorMessage property of each validation control. If the ErrorMessage property of the validation control is not set, no error message is displayed for that validation control.

Properties

Property	Description
DisplayMode	How to display the summary. Legal values are: • BulletList • List • SingleParagraph
EnableClientScript	A Boolean value that specifies whether client-side validation is enabled or not
Enabled	A Boolean value that specifies whether the validation control is enabled or not
ForeColor	The fore color of the control
HeaderText	A header in the ValidationSummary control
Id	A unique id for the control
Runat	Specifies that the control is a server control. Must be set to "server"
ShowMessageBox	A Boolean value that specifies whether the summary should be displayed in a message box or not
ShowSummary	A Boolean value that specifies whether the ValidationSummary control should be displayed or hidden

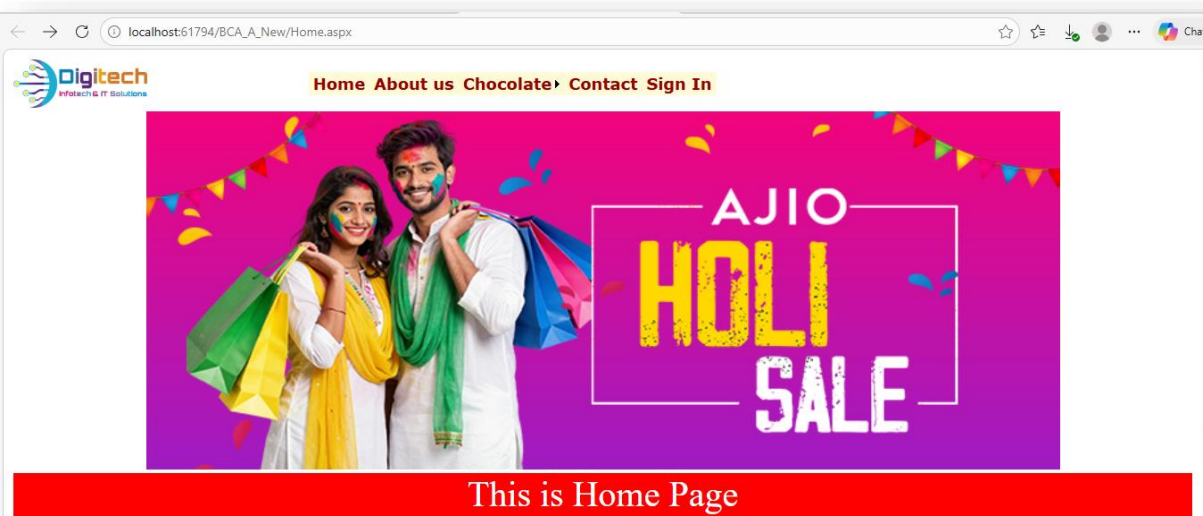
Q.-4. Explain AdRotator Control in detail.

- ❑ The AdRotator control is used to display a sequence of ad images.
- ❑ This control uses an XML file to store the ad information. The XML file must begin and end with an <Advertisements> tag. Inside the <Advertisements> tag there may be several <Ad> tags which defines each ad.
- ❑ The predefined elements inside the <Ad> tag are listed below:

Element	Description
<ImageUrl>	Optional. The path to the image file
<NavigateUrl>	Optional. The URL to link to if the user clicks the ad
<AlternateText>	Optional. An alternate text for the image
<Keyword>	Optional. A category for the ad
<Impressions>	Optional. The display rates in percent of the hits

Properties

Property	Description
AdvertisementFile	Specifies the path to the XML file that contains ad information
AlternateTextField	Specifies a data field to be used instead of the Alt text for an ad
ImageUrlField	Specifies a data field to be used instead of the ImageURL attribute for an ad
KeywordFilter	Specifies a filter to limit ads after categories
NavigateUrlField	Specifies a data field to be used instead of the NavigateUrl attribute for an ad
Runat	Specifies that the control is a server control. Must be set to "server"
Target	Specifies where to open the URL

Advertisement No. 1.**Advertisement No. 2.**

Q.-5 What is Login Control? Explain in detail.**The Login Control**

- ❑ The Login control displays a user interface for user authentication.
- ❑ The Login control contains text boxes for the user name and password and a check box that allows users to indicate whether they want the server to store their identity using ASP.NET membership and automatically be authenticated the next time they visit the site.
- ❑ The Login control has properties for customized display, for customized messages, and for links to other pages where users can change their password or recover a forgotten password. The Login control can be used as a standalone control on a main or home page, or you can use it on a dedicated login page.
- ❑ The “Login” control in its simplest form will look like the following:
`<asp:Login id="Login1" runat="server" />`
- ❑ The “Login” control displays text boxes that let the user enter a user name and password.
- ❑ If the fields are filled in, the “Login” control uses the membership provider to look up the user name and password in the membership database.
- ❑ If the user name and password are valid, the user is logged in and the page requested is displayed else if it is not valid, an error message is displayed and the user will not be logged in to show the requested page.
- ❑ If you use the Login control with ASP.NET membership, you do not need to write code to perform authentication.
- ❑ However, if you want to create your own authentication logic, you can handle the Login control's Authenticate event and add custom authentication code.

Attribute	Description
id	The “ID” for the Login control.
runat	runat="Server" is needed for all ASP.NET server controls.
CreateUserIconUrl	The URL of an image used as a link to the register new user page.
CreateUserText	The text displayed as a link to the register new user page.
CreateUserUrl	The URL to the register new user page.
DestinationPageUrl	The URL of the page for successful log in. If you don't specify this attribute, the page which the user was on before getting to this page is displayed.
DisplayRememberMe	A checkbox, which is a Boolean, to choose whether the Login Control should automatically let the user save his info by saving a cookie and avoid re-logging in.
FailureText	Is the text that will be displayed if the Log in information is not valid. If you do not change this message, a default message “Your login attempt has failed. Please try again” is displayed.
InstructionText	A text displayed underneath the title text providing the user with login information. If you do not change this attribute, the default is an empty string.
LoginButtonText	The Login button text.

LoginButtonType	The button type for the Login button. You can specify "Button", "Link", or "Image". (If you specify "Image", then you should also specify the "LoginButtonImageUrl" attribute.)
Orientation	This specifies the "Horizontal" or "Vertical" layout of the login control. If you do not change this attribute, the default is "Vertical".
PasswordLabelText	The text for the "Password" field.
PasswordRecoveryIconUrl	The URL for the image used as a link to the recover a lost password page.
PasswordRecoveryText	The text for the recover a lost password page.
PasswordRecoveryUrl	The URL for the recover a lost password page.
RememberMeText	Remember Me check box's text.
TextLayout	Here the position of the labels for the username and password textboxes are set, "TextOnLeft" for the labels to appear on to the left side of the text boxes or "TextOnTop" for the label to appear above the text boxes.
TitleText	The text for the top of the Login control.
UserNameLabelText	The text for the Username label.

- ❑ The LoginView control allows you to display different information to anonymous and logged-in users. The control displays one of two templates: the AnonymousTemplate or the LoggedInTemplate.
- ❑ In the templates, you can add markup and controls that display information appropriate for anonymous users and authenticated users, respectively.
- ❑ The LoginView control also includes events for ViewChanging and ViewChanged, which allow you to write handlers for when the user logs in and changes status.

The LoginStatus Control

- ❑ The LoginStatus control displays a login link for users who are not authenticated and a logout link for users who are authenticated.
- ❑ The login link takes the user to a login page. The logout link resets the current user's identity to be an anonymous user.
- ❑ You can customize the appearance of the LoginStatus control by setting the LoginText and LoginImageUrl properties.
- ❑ For example:
 - ❑ If the user is logged in, a link is displayed to Logout
 - ❑ If the user is not logged in, a link is displayed to Login

- ❑ The “LoginStatus” control in its simplest form will look like the following:
`<asp:LoginStatus ID="LoginStatus1" runat="server" />`
- ❑ you have the ability to customize your “LoginStatus” control by using any of the optional attributes listed below.

Attribute	Description
Id	The “ID” for the “LoginStatus” control
Runat	runat=“Server” is needed for all ASP.NET server controls.
LoginImageUrl	The URL for the “Login” link image.
LoginText	The text for the “Login” link.
LogoutAction	The action to do after the user logs out. This can be specified to “Redirect” to redirect the user to a page in the “LogoutPageUrl” attribute, or “RedirectToLoginPage” to redirect the user to the login page, or “Refresh” to refresh the current page.
LogoutImageUrl	The URL for the “Logout” link.
LogoutPageUrl	The URL for the redirect page after a user logs out if the “LogoutAction” attribute specifies “Redirect”.
LogoutText	The text for the “Logout” link.

The LoginName Control

- ❑ “LoginName” control is used to display the user’s username which the user logged in to the web application from. If a user is not currently logged into the web application, the “LoginName” control does not display anything.
- ❑ The “LoginName” control in its simplest form will look like the following:
`<asp:LoginName ID="LoginName1" runat="server" />`
- ❑ If you want a custom message such as a welcome message to be displayed in front of the Username, you can do the following:
`<asp:LoginName ID="LoginName1" runat="server" FormatString="Welcome, {0}" />`
- ❑ The “Welcome” text will be added as a prefix to the Username if the user has logged in, and if no user is logged in nothing is displayed.

The PasswordRecovery Control

- ❑ The PasswordRecovery control allows user passwords to be retrieved based on the e-mail address that was used when the account was created. The PasswordRecovery control sends an e-mail message containing a password to the user.
- ❑ You can configure ASP.NET membership to store passwords using non-reversible encryption. In that case, the PasswordRecovery control generates a new password instead of sending the original password to the user.
- ❑ You can also configure membership to include a security question that the user must answer to recover a password. If you do, the PasswordRecovery control asks the question and checks the answer before recovering the password.
- ❑ The “PasswordRecovery” control in its simplest form will look like the following:
`<asp:PasswordRecovery ID="PasswordRecovery1" runat="server">
</asp:PasswordRecovery>`

- ❑ The new password can be sent to the user's email account. The "PasswordRecovery" control can send a confirmation e-mail to the new user using two methods:
- ❑ In the "<MailDefinition>" child element
- ❑ The body of the e-mail message will be taken from the file "BodyFile.txt" in the "BodyFileName" attribute.
- ❑ This ".txt" file can include "<%UserName%>" and "<%Password%>" so you can put the user's account name and password into the email.
- ❑ You have the ability to customize your "PasswordRecovery" control by using any of the optional attributes listed below.

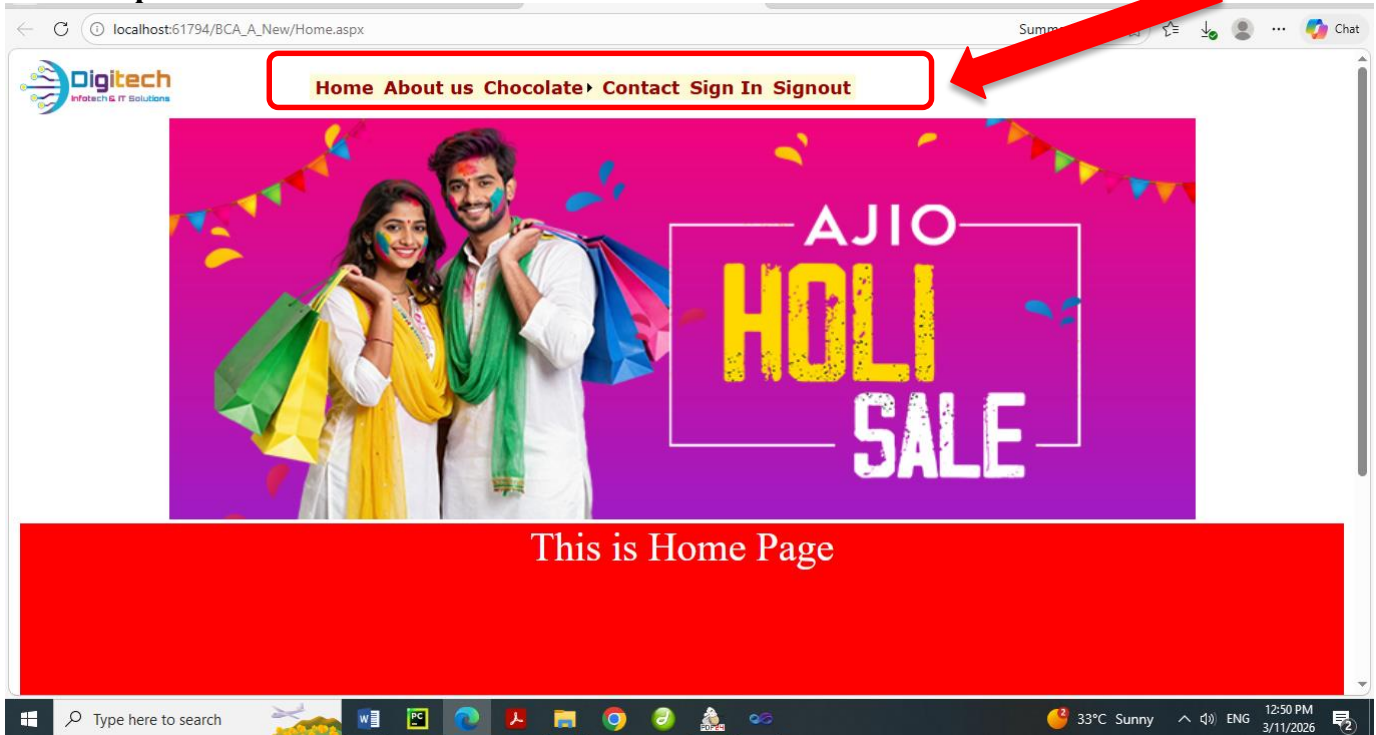
Attribute	Description
Id	The "ID" for the "PasswordRecovery" control
Runat	runat="Server" is needed for all ASP.NET server controls.
AnswerLabelText	The text for the "Answer" label field
GeneralFailureText	The text for the "Failure" text field if the password could not be retrieved.
QuestionFailureText	The text for "Question failure", that is if the user gives a wrong answer for the secret question.
QuestionInstructionText	The text for the instructions in the Question's answer part.
QuestionLabelText	The text for the "Secret Question" label.
QuestionTitleText	The text for the title area for the secret question.
SubmitButtonImageUrl	The URL for the image of the Submit button.
SubmitButtonText	The text for the Submit button.
SubmitButtonType	The type of the Submit button can be "Button", "Link", or "Image".
SuccessPageUrl	The URL for the Success page if the password has been recovered.
SuccessText	The text for the message if the password retrieval is successful, and this is not displayed if the "SuccessPageUrl" is provided.
TextLayout	Here the position of the labels for the username and password textboxes are set. "TextOnLeft" for the labels to appear on to the left side of the text boxes or "TextOnTop" for the label to appear above the text boxes.
UserNameFailureText	The text for incorrect username.
UserNameInstructionText	The text for the instructions for the Username request area.
UserNameLabelText	The text for the "Username" label field.
UserNameTitleText	The text for the title in the Username request area.

Q.6 What is Navigation control? Explain in detail.**1) Menu:**

- ❑ The Menu control is another Navigation control, which is also used to display site navigation information.
- ❑ The Menu control can, however, display the site structure vertically as well as horizontally. It can be used as a databound control, for example, binding the menu control with a SiteMapDataSource control.
- ❑ It can also retrieve the data to be displayed from the items that are added to the Items collection of the Menu control at runtime.
- ❑ The Menu control supports binding data with hierarchical data source, such as XML files.
- ❑ You can also add the database items to the Menu control at runtime using the Items collection of the Menu control.
- ❑ The Menu control contains two types of menus:
 - ❑ **Static menu:-** Refers to the type of menu that is displayed completely on the screen with all the menu options on the screen. The entire structure of the static menu, including the parent menu items and their sub menus, is visible.
 - ❑ **Dynamic menu:-** Refers to the type of menu that contains static part, which is displayed on the screen, as well as the dynamic part, which appears when user passes the mouse pointer over the static part of the menu.

Properties:

Property	Description
Orientation	Means that you can set this property to horizontal or display menu either at the top or at the bottom of the page. The default orientation value is Vertical.
StaticItemFormatString	Means that you can set this property to display the value for the static menu.
DynamicItemFormatString	Means that you can set this property to display the value for the dynamic menus.
StaticPopOutImageUrl	Means that you can set the replacement image for StaticPopOutImage. This means that the menu items can be hovered over to display all of its child items. The default image will appear dynamically and continue to be used for menu items.
Static/DynamicMenuStyle	Controls properties for all MenuItem property. It also controls properties such as HorizontalPadding and VerticalPadding.
DynamicMenuItemStyle	Controls the style of the individual MenuItem property. It also includes style properties such as ItemSpacing, VerticalPadding, and HorizontalPadding.
DynamicSelectedStyle	Sets the style of the individual nodes in the TreeView control that are currently selected.
DynamicHoverStyle	Sets the style properties of the individual MenuItem property that is currently selected.

Example:**2) Tree View:**

- ❑ The TreeView controls is used for logically displaying the data in a hierarchical structure, similar to Windows explorer.
- ❑ You can use this control when you need to display the navigation menu for displaying the file and folders on the Web page.
- ❑ You can display an Extensible Markup Language (XML) document and database records in a tree structure.
- ❑ To work with the TreeView control at runtime, you can programmatically access the TreeView Web server control.
- ❑ The TreeView Web server control is used to create tree-based hierarchical structure, set properties, and populate nodes.
- ❑ The TreeView control consists of the TreeView container and TreeNode properties.
- ❑ There can be multiple nodes in the tree view structure, which can be nested.
- ❑ The TreeNode property has a Text property that displays the text for the node.
- ❑ You can access the TreeView control directly from the Navigation tab in the Toolbox.
- ❑ You can add the TreeView control to your Web page simply by either double clicking the TreeView control in the Toolbox or by dragging the control from the Toolbox to the Design view of the Default.aspx page.
- ❑ You can also add the TreeView control on the Web page by adding the following code snippet in the <form> element of your web page:

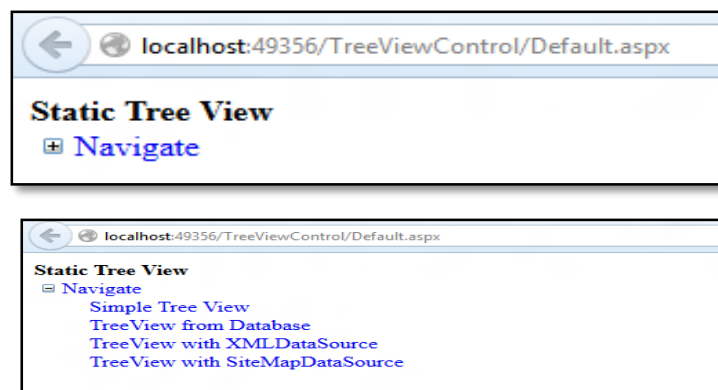
```
<asp:TreeView ID="TreeView1" runat="server">
</asp:TreeView>
```
- ❑ The content for the TreeView control can be added in the following ways:
 - ❑ By adding programmatically at runtime
 - ❑ By dynamically loading from an external data source, such as an XML file.

Adding Node to a TreeView Control Dynamically:

- ❑ The TreeView control can load the data dynamically at runtime.
- ❑ For this, you have to set the property of the TreeView nodes dynamically.
- ❑ The TreeView control has a Nodes property known as TreeView.Nodes.
- ❑ You can add nodes to the TreeView control by using the add method of the TreeView.Nodes property.

Adding Node to a TreeView Control using the DataSource Property:

- ❑ You can also add data to the TreeView control by using the DataSource property.
- ❑ In fact, this is a more flexible and easy-to-use property for binding data.
- ❑ By setting the DataSourceID property of the TreeView control to the desired file, you can load the content of the TreeView control from the specified file.
- ❑ The target file can be an XML file, a sitemap, or you can even populate a tree view from the database.
- ❑ After an appropriate data source is set for the TreeView control, the TreeView control displays the information in a tree structure.
- ❑ Using the DataSource property is a flexible approach for the TreeView control as you can easily change the text of any of the nodes by changing the content of the underlying file.
- ❑ You can only bind the TreeView control to the XmlDataSource and SiteMapDataSource control using Add Data Source Wizard.

Example:**3) Sitemap Path:**

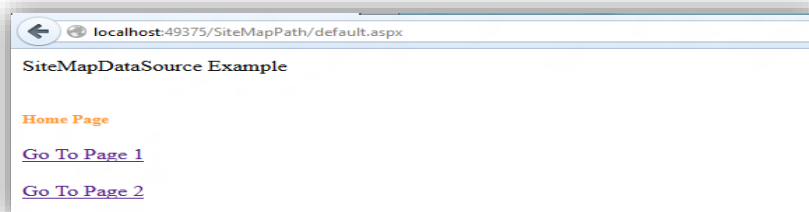
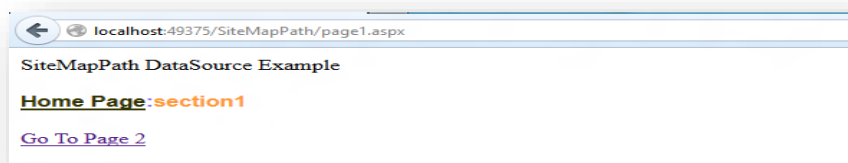
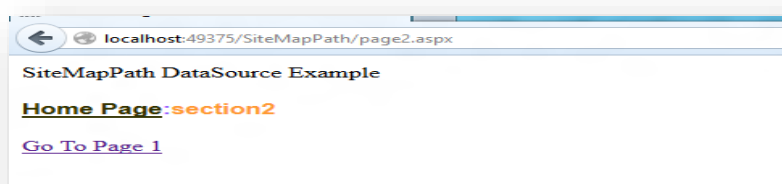
- ❑ The SiteMapPath control is also known as the breadcrumb Navigation control.
- ❑ Breadcrumb allows you to display the current page's context within the entire structure of a Web site. It also allows you to display links as a path back to the home page.
- ❑ It displays information regarding the user's current page along with the entire hierarchy of the pages, thereby enabling the user to navigate back to some other pages in the hierarchy.
- ❑ The SiteMapPath control does not use the SiteMapDataSource control to display data. It relies on a SiteMapProvider to retrieve the data. It contains XML file web.sitemap, which contains URLs and other information about the pages to be displayed.

Data retrieval using the SiteMapPath control:

- ❑ The SiteMapPath control uses the SiteMapProvider to fetch the data and display it on the Web form.
- ❑ As you have seen earlier, both the Menu and TreeView controls rely on the SiteMapDataSource control.
- ❑ The SiteMapPath control displays only the data whose URL is present in the specified SiteMapProvider.

Properties:

Property	Description
ParentLevelsDisplayed	Set to -1 by default, which means that all parent levels in the XML file are displayed. You can set it to any integer values, such as 1, 2, or 3, to indicate the N-level of the hierarchy along with the current page.
PathDirection	Specifies the order in which the nodes of the navigation path are rendered. The default setting is such that it starts from the highest node in the current XML file and goes up to the current page.
PathSeparator	Set this property to : default value is >, you can set as semicolon
RenderCurrentNodeAsLink	Sets this property to true, so that the current node text on the Web page will appear as hyperlink and not as plain text.
ShowToolTips	Sets this property to false, so that the description text from the SiteMapProvider control is not displayed automatically.
NodeStyle	It is used to control the appearance of the text for all the nodes displayed by the SiteMapPath control.
CurrentNodeStyle	It is used to set the property of the node corresponding to the currently displayed Web page.
RootNodeStyle	It is used to set the style for the absolute root in the hierarchy of the SiteMapProvider control.
PathSeparatorStyle	It is used to control the appearance of the path separator that appears between navigation path nodes.

Example:**Home Page****Page: 1****Page: 2**

Q.7 How to Upload File in Asp.Net C#? Explain with properties.

- ❏ The FileUpload control allows the user to browse for and select the file to be uploaded, providing a browse button and a text box for entering the filename.
- ❏ Once, the user has entered the filename in the text box by typing the name or browsing, the SaveAs method of the FileUpload control can be called to save the file to the disk.
- ❏ The basic syntax of FileUpload is:

```
<asp:FileUpload ID= "Uploader" runat = "server" />
```

- ❏ The FileUpload class is derived from the WebControl class, and inherits all its members. Apart from those, the FileUpload class has the following read-only properties:

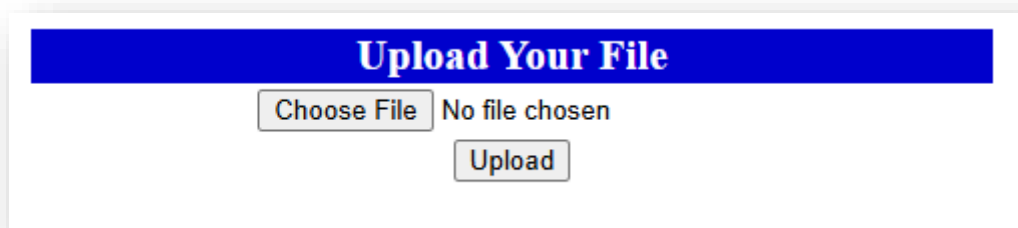
Properties	Description
FileBytes	Returns an array of the bytes in a file to be uploaded.
FileContent	Returns the stream object pointing to the file to be uploaded.
FileName	Returns the name of the file to be uploaded.
HasFile	Specifies whether the control has a file to upload.
PostedFile	Returns a reference to the uploaded file.

- ❏ The posted file is encapsulated in an object of type HttpPostedFile, which could be accessed through the PostedFile property of the FileUpload class.
- ❏ The HttpPostedFile class has the following frequently used properties:

Properties	Description
ContentLength	Returns the size of the uploaded file in bytes.
ContentType	Returns the MIME type of the uploaded file.
FileName	Returns the full filename.
InputStream	Returns a stream object pointing to the uploaded file.

Example

- ❏ The following example demonstrates the FileUpload control and its properties. The form has a FileUpload control along with a save button and a label control for displaying the file name, file type, and file length.
- ❏ In the design view, the form looks as follows:



Q.8 What is Image Control? Explain with an example.**Image Control:**

- ❑ The image control is a standard Web server Control, which is used to display an image on a Web page.
- ❑ This control exists within the System.Web.UI.WebControls namespace.
- ❑ You set uniform Resource Locator of the image with the ImageUrl property of the image control.
- ❑ The alignment of the image in relation to other elements on the Web page is specified by setting the ImageAlign property.
- ❑ The inheritance hierarchy for the Image class is as follow:

```

System.Object
    System.Web.UI.Control
        System.Web.UI.WebControls.WebControl
            System.Web.UI.WebControls.Image
  
```

Properties:

Property	Description
AlternateText	Alternate text for the image required for accessibility.
DescriptionUrl	A link to a page that contains a detailed description of the image (required to make a complex image accessible).
GenerateEmptyAlternateText	Set the AlternateText property to an empty string.
ImageAlign	Align the image relative to other HTML elements in the page. Possible values are AbsBottom, AbsMiddle, Baseline, Bottom, Left, Middle, NotSet, Right, TextTop, and Top.
ImageUrl	Specify the URL to the image.

Example:**Default.aspx**

```

Partial Class _Default
    Inherits System.Web.UI.Page

    Protected Sub Button1_Click(ByVal sender As Object, ByVal e As
        System.EventArgs) Handles Button1.Click
        Image1.Visible = True
    End Sub
End Class
  
```

Q.9 What is ImageButton Control? Explain with an example.**ImageButton Control:**

- ❑ The ImageButton control is a Button control that displays an image instead of text.
- ❑ The ImageButton control exists within the System.Web.UI.WebControls namespace.
- ❑ This control is specifically useful when you want to display an image that responds to mouse click and can initiate various actions, such as redirecting the user to the next page, depending on part of the image that you click.
- ❑ The inheritance hierarchy of the ImageButton class is as follow:

System.Object

System.web.UI.Control

System.web.UI.WebControls.webControl

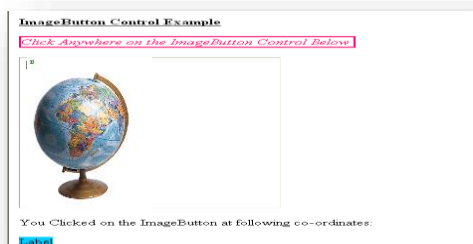
System.web.UI.WebControls.Image

System.web.UI.WebControls.ImageButton

- ❑ The ImageButton class inherits the Image class and has no non-inherited methods.

Properties:

Property	Description
AlternateText	Obtains or sets the alternate text displayed in the Image control when the image is not available. Browsers that support the ToolTips feature display this text as a tool tip.
DescriptionUrl	Obtains or sets the location to a detailed description for the image.
Enabled	Obtains or set a value indicating whether the control is enabled.
Font	Obtains the font properties for the text associated with the control.
GenerateEmptyAlternateText	Obtains or set a value indicating whether the control generates an alternate text attribute for an empty string value.
ImageAlign	Obtains or sets the alignment of the image control in relation to other controls on the Web page
ImageUrl	Obtains or set the URL that provides the path to an image to display in the Image control.

Example:**Default.aspx**

```
Partial Class _Default
    Inherits System.Web.UI.Page

    Protected Sub ImageButton1_click(ByVal sender As Object, ByVal e As
System.Web.UI.ImageClickEventArgs) Handles ImageButton1.Click
        Label4.Text = e.X.ToString() & " , " & e.Y.ToString()
    End Sub
End Class
```